

Image Compression using the Discrete Cosine Transform

8 × 8 Block DCT, Coefficient Selection and PSNR Analysis in MATLAB

1. Aim

To compress a grayscale image in MATLAB using the **Discrete Cosine Transform (DCT)** — the same transform used in JPEG — by keeping only the most important DCT coefficients of each 8 × 8 block, and to study the trade-off between the **compression ratio** and the **image quality (PSNR)**.

2. Introduction

A digital image contains a very large amount of data, which is expensive to store and slow to transmit. **Image compression** reduces this data by removing the parts of the image that the eye does not notice much, while keeping the image looking almost the same.

The most successful method — used in the **JPEG** standard — is **transform coding** with the **Discrete Cosine Transform**. The DCT packs most of the image's energy into a few coefficients, so the rest can be thrown away with very little visible loss. This is a core topic of **image transforms and compression** in the Machine Vision syllabus.

Efficient image compression is essential for digital cameras, web and mobile images, video streaming and medical/satellite image storage, so this work links to **SDG 9 (Industry, Innovation and Infrastructure)**.

3. Theory

3.1 Transform Coding

Instead of compressing the pixels directly, the image is first changed into a **transform domain** where its energy is concentrated into a few values. The small, unimportant values are then discarded. The DCT is chosen because it has excellent **energy compaction** for natural images and uses only real numbers, which makes it simple and fast.

3.2 The 2-D DCT and 8 × 8 Blocks

Like JPEG, the image is split into **8 × 8 blocks** and each block is transformed separately. The forward 2-D DCT of an N × N block f(x,y) is:

$$C(u,v) = \alpha(u)\alpha(v) \sum_x \sum_y f(x,y) \cdot \cos[(2x+1)u\pi / 2N] \cdot \cos[(2y+1)v\pi / 2N]$$

with $\alpha(0) = \sqrt{(1/N)}$ and $\alpha(k) = \sqrt{(2/N)}$ for $k > 0$. The coefficient **C(0,0)** (top-left) is the **DC term** — the average brightness of the block — and the coefficients further right/down represent higher and higher spatial frequencies (fine detail).

3.3 Energy Compaction and Coefficient Selection

For a natural image almost all of a block's energy ends up in the **top-left (low-frequency) coefficients**, while the high-frequency ones are nearly zero. Compression keeps only the first **k** coefficients of each block, scanned in **zig-zag order** (low to high frequency), and sets the rest to zero. Keeping fewer coefficients gives a **higher compression ratio** but lower quality. The compression ratio is approximately:

$$\text{Compression Ratio} \approx 64 / k$$

3.4 Quality Measures (MSE and PSNR)

The quality of the reconstructed image is measured against the original using the **Mean Squared Error (MSE)** and the **Peak Signal-to-Noise Ratio (PSNR)**. A higher PSNR means a better (closer) image:

$$\text{MSE} = (1/MN) \sum [f - \hat{f}]^2 \quad \text{PSNR} = 10 \cdot \log_{10}(255^2 / \text{MSE}) \text{ dB}$$

4. Methodology

Implemented in MATLAB using the Image Processing Toolbox (dct2, idct2, blockproc). The steps are:

1. Read the input image and convert it to grayscale.
2. Split the image into 8×8 blocks and take the 2-D DCT (dct2) of each block.
3. Keep only the first k coefficients of each block (zig-zag order) and set the rest to zero.
4. Reconstruct the image with the inverse DCT (idct2).
5. Compute the compression ratio, MSE and PSNR, and repeat for several values of k.

Data flow: Image $\rightarrow 8 \times 8$ blocks $\rightarrow$ DCT $\rightarrow$ keep k coefficients $\rightarrow$ inverse DCT $\rightarrow$ reconstructed image $\rightarrow$ measure CR, MSE, PSNR.

5. MATLAB Code

blockproc applies the same DCT-keep-inverse operation to every 8×8 block, and the main loop repeats it for each value of k. The helper zigzagmask builds an 8×8 logical mask that keeps the first k zig-zag coefficients. Keep **cameraman.tif** (or any image) in the same folder.

```
%% Image Compression using the Discrete Cosine Transform (DCT)
% Keeps the first k DCT coefficients of each 8x8 block (JPEG idea)

clc; clear; close all;

% STEP 1 - Read the image and convert to grayscale
I = imread('cameraman.tif');
if size(I,3) == 3, I = rgb2gray(I); end
I = double(I);

ks = [1 4 8 16]; % number of DCT coefficients kept per block

for idx = 1:numel(ks)
    k = ks(idx);
    mask = zigzagmask(8, k); % 8x8 mask: 1 = keep, 0 = drop

    % forward DCT -> keep k coeffs -> inverse DCT, on every 8x8 block
    fun = @(b) idct2( dct2(b.data) .* mask );
    R = blockproc(I, [8 8], fun);

    % quality measures
    mse = mean( (I(:) - R(:)).^2 );
    psnr = 10 * log10( 255^2 / mse );
    fprintf('Keep %2d/64 CR %2d:1 MSE %7.2f PSNR %.2f dB\n', ...
        k, 64/k, mse, psnr);

    figure; imshow(uint8(R));
    title(sprintf('Keep %d/64 (CR %d:1, PSNR %.1f dB)', k, 64/k, psnr));
end

% ---- helper: 8x8 mask keeping the first k zig-zag coefficients ----
function m = zigzagmask(n, k)
    order = zeros(n*n, 2); idx = 1;
    for s = 0:2*n-2
```

```

xs = s:-1:0;
if mod(s,2)==1, xs = 0:s; end      % alternate scan direction
for x = xs
    y = s - x;
    if x < n && y < n
        order(idx,:) = [x y]; idx = idx + 1;
    end
end
end
m = false(n);
for i = 1:k, m(order(i,1)+1, order(i,2)+1) = true; end
end

```

6. The Image in the Transform Domain

Figure 1 shows the original image and its 8×8 block DCT (log magnitude). In every small block the energy is concentrated in the **top-left corner** (the bright dot of each cell), while the rest of the block is almost black — this is the energy compaction that makes compression possible.

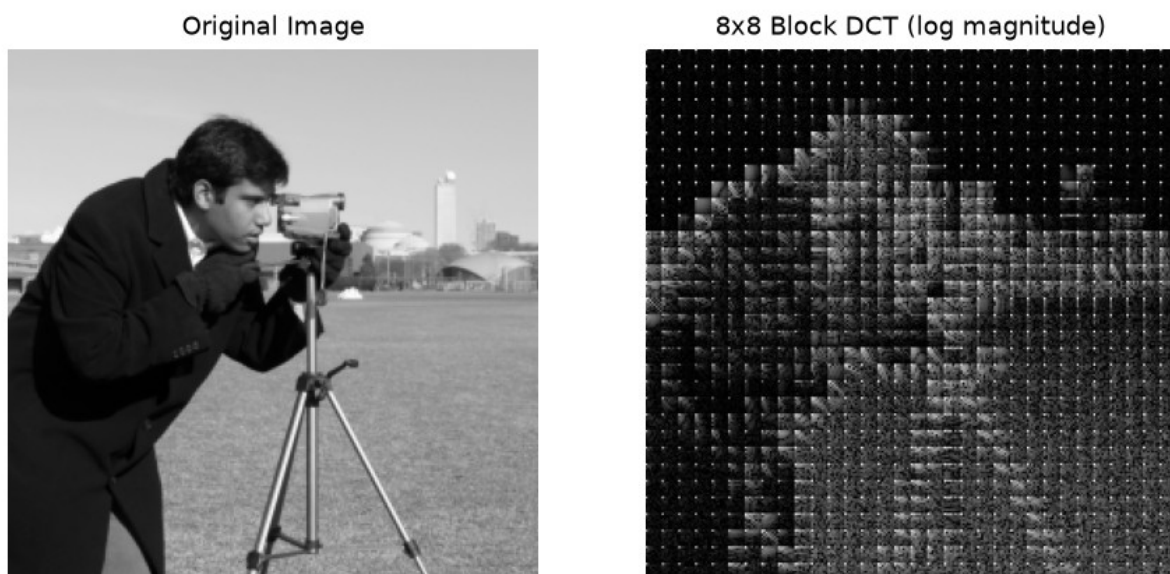


Figure 1: Original image (left) and its 8×8 block DCT, log magnitude (right).

7. Results and Output Images

Figure 2 shows the image rebuilt after keeping only $k = 1, 4, 8$ and 16 coefficients per block. With $k = 1$ (only the DC term) the image is very blocky at a 64:1 ratio; by $k = 8$ it already looks close to the original at 8:1; and $k = 16$ (4:1) is almost indistinguishable from the original. Figure 3 plots the resulting quality against the compression ratio.

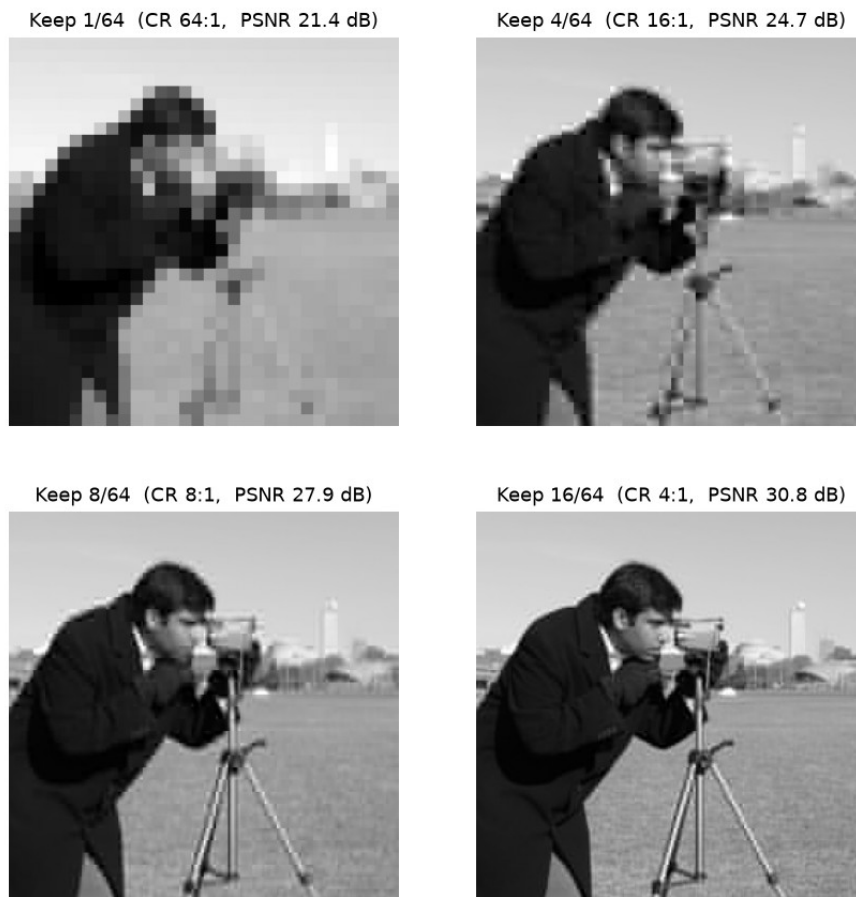


Figure 2: Reconstructed images for different numbers of kept DCT coefficients.

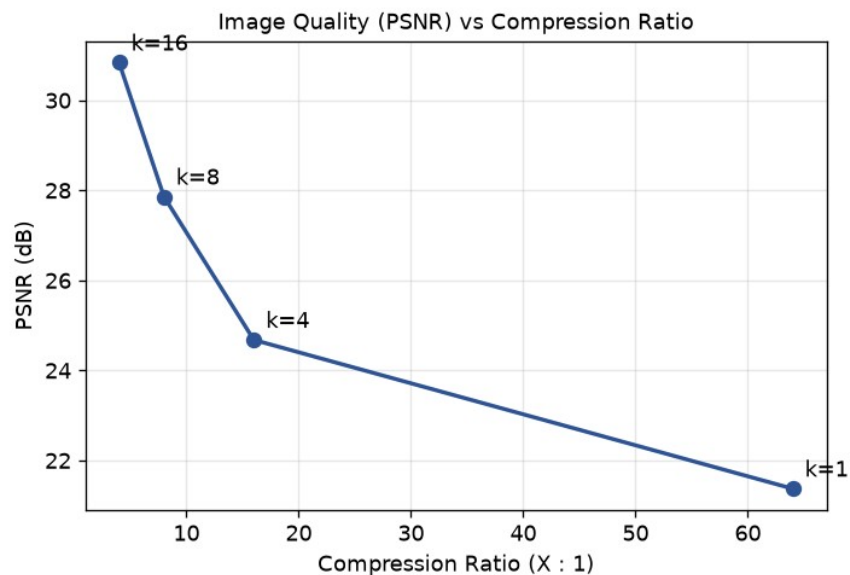


Figure 3: Image quality (PSNR) falls as the compression ratio rises.

8. Quantitative Results

Each 8×8 block has 64 DCT coefficients; "coefficients kept" is how many of those were retained. PSNR is measured against the original 256×256 image.

Coefficients Kept (of 64)	Compression Ratio	MSE	PSNR (dB)
1 (DC only)	64 : 1	474.38	21.37
4	16 : 1	221.59	24.68
8	8 : 1	106.52	27.86
16	4 : 1	53.65	30.84

9. Result Analysis

- As **more coefficients are kept**, the compression ratio falls (64:1 → 4:1) but the PSNR rises (21.4 dB → 30.8 dB) — there is a direct **trade-off between compression and quality**.
- Keeping only the **DC coefficient** ($k = 1$) gives the smallest file (64:1) but a visibly **blocky** image, because each 8×8 block becomes a single flat gray value.
- Keeping about **8 coefficients** already gives a good image (PSNR ≈ 28 dB) at a useful 8:1 ratio, which shows the DCT's strong energy compaction — only $\frac{1}{8}$ of the coefficients carry almost all of the visible detail.

A PSNR above about 30 dB is usually considered good quality, which is reached here at $k = 16$ (4:1). So a **moderate number of low-frequency DCT coefficients is enough** to rebuild the image well, which is exactly why DCT-based JPEG compression is so effective.

10. Advantages and Limitations

Aspect	Details
Advantages	Excellent energy compaction; real-valued and fast; block-based so simple to implement; adjustable quality/compression; the basis of the JPEG standard.
Limitations	At high compression the 8×8 blocks become visible ("blocking artifacts"); it is lossy (original cannot be recovered exactly); less efficient than wavelets (JPEG 2000) at very high compression.

11. Applications

The DCT is used in **JPEG image compression, MPEG and H.26x video compression**, image and video storage in cameras and phones, web and streaming media, video conferencing, and the storage/transmission of **medical and satellite images** — anywhere image data must be made smaller with little visible loss.

12. Conclusion

Image compression was implemented in MATLAB using the **8×8 block Discrete Cosine Transform**, keeping only the most important low-frequency coefficients of each block. The experiment clearly showed the **trade-off between compression ratio and quality**: 64:1 compression gave a blocky 21.4 dB image, while 4:1 gave a near-perfect 30.8 dB image. Because the DCT concentrates image energy into a few coefficients, a small number of

them is enough to rebuild the image well — which is why the DCT is the heart of JPEG. The project meets CO-1 and CO-4 and supports SDG 9 (Industry, Innovation and Infrastructure).

13. Course Outcome and SDG Mapping

CO / SDG	How this project addresses it
CO-1	Implements an image-transform / compression algorithm (block DCT) in MATLAB.
CO-4	Uses the mathematical basis of the Discrete Cosine Transform and PSNR/MSE quality measures.
SDG 9	Supports digital infrastructure — efficient image storage and transmission (JPEG, streaming).

14. References

- [1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, Addison-Wesley.
- [2] N. Ahmed, T. Natarajan and K. R. Rao, "Discrete Cosine Transform," *IEEE Trans. Computers*, 1974.
- [3] G. K. Wallace, "The JPEG Still Picture Compression Standard," *Communications of the ACM*, 1991.
- [4] MATLAB Documentation — Image Processing Toolbox: dct2, idct2, blockproc.